

Python 入门（教科书版）

打印提示：A4、实际大小（100%）、**关闭浏览器"页眉和页脚"**——本套教材的 PDF 版已自带页码，重复勾选会产生双页码；需要保留彩色底色时，在打印选项里开启"背景图形"。

目录

使用说明 配色约定 · 范围说明 · 与 Linux 三连的关系

第 0 章 Python 与 AI 运维的关系 (约 1h)

第 1 章 起步: print、变量、类型 (约 1.5h)

第 2 章 控制流: if、for、while (约 2h)

第 3 章 数据结构: list、dict、tuple、set (约 2.5h)

第 4 章 函数: def、参数、返回值 (约 2h)

第 5 章 模块与导入: import、os、time (约 1.5h)

第 6 章 文件操作: open/with、json (约 2h)

第 7 章 异常处理: try/except/finally (约 1.5h)

第 8 章 网络请求: requests (约 2h)

第 9 章 命令行参数: argparse (约 1.5h)

第 10 章 日志: logging (约 1h)

第 11 章 进程调用: subprocess (约 1.5h)

附录 A 章末作业参考答案 (含思考过程)

附录 B 术语表 (中英对照)

附录 C 学习路线图 (第 0~11 章共 12 章顺序 + 每章耗时)

附录 D 函数与模块速查

附录 E venv 与 pip (项目独立环境 / 依赖清单)

使用说明

📅 学习日志：每天把敲过的代码、报错与心得记进学习日志——坚持记录本身就是在练习“写下来”。

本书配色约定（全书一致）：

样式	含义
深蓝粗体	Python 关键字（def / if / for / import 等，语法骨架）
绿色	函数名、模块名（print、len、os 等）
深红	字符串（引号里的内容）
灰色斜体	注释（# 后面的说明）
灰色	程序的输出（运行结果）
蓝色	解读、分析（帮你理解“为什么”）
深紫	参数、变量值（数据，不是语法）
红色	危险警告、报错

实验闭环：涉及文件覆盖、重命名、联网或系统命令的示范与作业，先在练习目录记录文件状态；运行后核对输出、退出码和生成文件；结束后删除临时文件或恢复原文件。遇到异常保留 traceback 和输入条件。

范围说明：本书只教“用 Python 做自动化运维”够用的内容，目标 = 写出脚本 1（批量文件重命名）、脚本 2（天气查询器），并为王牌项目 server-health-check 打基础。面向对象进阶、异步、类型注解等超纲内容一律不进本书。

学习顺序：本书在 Linux 三连之后开始。主线是本书；语法卡壳时可回到对应章节重读。

环境：本套资料以 **Ubuntu 26.04 + Python 3.14** 为基准（26.04 自带 Python 3.14，无需额外安装；详见《Linux 入门》的“环境口径”）。全书代码都在电脑或平板 Termux 的终端里敲。

第 0 章 • Python 与 AI 运维的关系

本章目标（学完能回答）：① Python 是什么、为什么 AI 运维要用它；② 会装好环境并跑出第一行代码；③ 知道 Python 和 Shell 怎么分工。

本章新知识点：print()（打印）、两种运行方式（交互式 / 脚本文件）。

0.1 Python 是什么

Python 是一门**编程语言**——和你学过的 Shell 一样，都是“给电脑下命令”。区别在于：Shell 擅长**系统操作**（ls、df、装软件），Python 擅长**逻辑处理**（判断、循环、处理数据）。

比喻：Shell 像你的**双手**——直接搬文件、开服务；Python 像你的**大脑**——“如果磁盘满 80% 就发警告、连续三次失败就重试”这类有逻辑的事，交给大脑。AI 部署运维 = 双手 + 大脑配合干活。

示范 1：验证环境

```
$ python3 --version
```

```
Python 3.14.4
```

看到 3.x 就是装好了。Ubuntu 自带，不用装。

示范 2：交互式模式跑第一行代码

```
$ python3
```

```
>>> print("Hello")
Hello
>>> exit()
```

输入 `python3` 回车后，提示符变成 `>>>`，这就是**交互式模式**：敲一行、马上出结果，适合试代码。敲 `exit()` 退出（就像 Linux 敲 `exit`）。

示范 3：脚本文件方式（正式工作方式）

```
$ nano hello.py # 写一行 print("Hello")
$ python3 hello.py
```

```
Hello
```

把代码存进 **.py 文件**（脚本）再整体运行，这是正式方式——代码能保存、能复用。交互式适合试，脚本适合写。**nano 用法**：进入编辑器写代码，写完按 **Ctrl+X → Y → 回车** 保存退出（nano 比 vim 友好，适合新手）。

0.2 学完本书能做出什么

本书学完你将写出两个正式脚本：

产出	长什么样
脚本 1：批量文件重命名	给一个目录里 100 张照片自动加日期前缀——你写过的手动改名操作，变成一条命令
脚本 2：天气查询器	敲 <code>python3 weather.py 杭州</code> ，程序自动联网拿天气并打印

这两件成品写完后，就打通了通往王牌项目 **server-health-check**（服务器健康巡检系统）的全部前置知识。

0.3 Python 与 Shell 的分工

	Shell	Python
强项	系统操作：文件、服务、权限	逻辑处理：判断、循环、算数据
怎么调用对方	脚本里直接敲命令	<code>subprocess</code> （第 11 章）调系统命令
举例	<code>df -h</code> 查磁盘	读 <code>df</code> 的输出，判断“超过 80% 就发警告”

一句总结：Shell 管系统层，Python 管逻辑层，第 11 章的 `subprocess` 让两者合流——这就是 AI 运维工程师每天的活。

0.4 易错点（这章的坑）

① `python` 和 `python3` 混用：有的系统 `python` 指向旧版，**一律用 `python3`**；② 在交互式里粘贴多行代码容易缩进错乱——多行代码用脚本文件；③ 忘写 `exit()` 退出交互式（`Ctrl+D` 也行）。

0.5 章末作业

作业（无代码，写下来）：“我认为 Python 将帮我完成的三件事。”

提示方向：自动化重复操作、调 API 拿数据、处理日志文件。写完记进学习日志——一段时间后回看，这就是你的起点。

 **对应练习册：暂无（先把 python3 跑通即可）**

0.6 这章你会了吗？

- ☐ 能说出 Python 和 Shell 的分工（系统层 vs 逻辑层）
- ☐ 会进交互式模式、会退出、会用脚本文件方式运行
- ☐ 能说出学完本书要交出的两个成品脚本

📖 相关手册：内置函数速查 → 见《Python 内置函数手册》第 1 组（print/input）。

第 1 章 · 起步：print、变量、类型

本章目标（学完能做什么）：能写“自我介绍卡”程序——程序打印出名字、年龄，还会算两位数倒序。这是所有 Python 代码的地基。

本章新知识点：`print`（打印）· `type`（查类型）· `len`（长度）· 变量 · `int` / `float` / `str` 三类型 · f-string · 七种运算。

1.1 `print()` 的两个细节：sep 和 end

`print()` 把内容打印到屏幕。默认多个内容之间用空格隔开（sep），结尾自动换行（end）。

示范 1：多个内容 + 自定义分隔符

```
print("a", "b", "c")
print("a", "b", "c", sep="-")
```

```
a b c
a-b-c
```

第二行指定了 sep（分隔符）为“-”。sep、end 这种写在括号里的“设置项”叫**关键字参数**——后面第 4 章会细讲，现在照抄用。

示范 2：end 取消换行

```
print("第一行", end="")
print("和第一行连着")
```

```
第一行和第一行连着
```

默认 `print` 结束会换行；指定 `end=""`（空字符串）就是不换行。这招在“循环里一行行拼进度条”时常用。

1.2 变量 = 贴标签的盒子

变量是装数据的**盒子**，标签是盒子名。`age = 19` 的意思：造一个盒子，贴标签“age”，把数字 19 放进去。以后写 `age` 就等于 19。

示范 1：赋值与重新赋值

```
name = "小明"
print(name)
name = "运维新人" # 重新赋值：盒子换新内容
print(name)
```

```
小明
运维新人
```

变量值**可以换**——这就是“变”量。和 Linux 的变量一样（`$HOME` 也是盒子），但 Python 里不用加 `$` 符号。

示范 2: type() 查盒子装的什么货

```
print(type(5))
print(type(5.0))
print(type("5"))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
```

`type()` (查类型) 是入门第一神器: 不确定一个值是什么类型, 就问它。三种基本类型: **int** 整数、**float** 小数、**str** 字符串 (引号包着的一切)。**"5"** 是字符串不是数字——类型不同, 能干的活不同 (1.5 会讲)。

1.3 字符串的操作: +、*、len、f-string

字符串 = 一串文字。两个操作符最常用: + 拼接、* 重复。

示范 1: 拼接与重复

```
print("你好" + "世界")
print("哈" * 3)
print(len("hello"))
```

```
你好世界
哈哈哈
5
```

`len()` (长度) 数字符个数。**"哈" * 3** 就是重复三次——和 Linux 的 `echo 哈{1..3}` 类似的想法, 不同写法。

示范 2: f-string 格式化 (日常最常用)

```
name = "小明"
age = 19
print(f"我叫{name}, 今年{age}岁")
n = 7
print(f"编号{n:03d}")
```

```
我叫小明, 今年19岁
编号007
```

f-string: 字符串前加字母 **f**, 里面用 `{变量}` 就能把变量的值塞进文字。和 Linux 的 `echo "我叫$name"` 完全一个思路——只是把 `$` 换成了 `{}`。本章作业全靠它。**花括号里的冒号 = 格式**: `{n:03d}` = 占 3 位、不足补零 (0=补零, 3=位数, d=整数) ——7 变成 007。以后生成"第 005 号"这种规范编号 (工号、订单号、报告序号) 全靠它。

示范 3: split 切割字符串（处理数据的起手式）

```

line = "2026-08-25 14:30:12 INFO 服务启动"
parts = line.split(" ")
print(parts)
print(parts[0])

```

```

['2026-08-25', '14:30:12', 'INFO', '服务启动']
2026-08-25

```

split()（切割）= 按指定符号把字符串切开，返回一个**列表**（列表第 3 章细讲，现在先记住"切成一片一片"）。这是日后解析日志、处理数据的**起手式**——一行日志切成了日期、时间、级别、消息四块，要哪块取哪块。

1.4 数字运算：七种运算符

运算符	含义	例子	结果
+	加	3 + 2	5
-	减	3 - 2	1
*	乘	3 * 2	6
/	真除法（带小数）	7 / 2	3.5
//	整除（砍掉小数）	7 // 2	3
%	取余（剩多少）	7 % 2	1
**	幂（几次方）	2 ** 3	8

示范 1: / 与 // 的区别（新手必看）

```

print(7 / 2)
print(7 // 2)
print(7 % 2)

```

```

3.5
3
1

```

/ 是**真除法**：7 除 2 就是 3.5，老实给小数；// 是**整除**：只要整数部分 3；% 是**取余**：7 个苹果分给 2 人，每人 3 个还剩 1 个——所以是 1。% 在运维里极常用（判断奇数偶数、轮询分组）。

示范 2: 类型不同不能乱加

```

print("我有" + 19 + "岁")

```

```

TypeError: can only concatenate str (not "int") to str

```

报错了！**字符串和数字不能直接相加**——类型不同。怎么改？两种办法：把数字转成字符串 `str(19)`（转字符串），或干脆用 f-string。报错读法见第 7 章，现在记住：类型必须对。

1.5 变量命名规则

三个规矩：① 只能由字母、数字、下划线组成，**不能数字开头**（2name 非法，name2 可以）；② 不能用 Python 关键字（if、for、while、def 等语法词）；③ 大小写敏感（name 和 Name 是两个变量）。**补充：**print 不是关键字，是内置函数名——语法上允许你拿它当变量名（print = 5 不报错），但这么干之后 print 就不能用了。所以**内置函数名也别占用**。

一个好习惯：名字要“见名知意”：age 年龄、disk_usage 磁盘使用率。多个词用下划线连接（叫蛇形命名）——和你 Linux 脚本里的习惯一致。

1.6 易错点（这章的坑）

① "5" + 5 类型不同报 TypeError；② 变量名 2name 非法；③ 变量名拼错报 NameError（name 'xxx' is not defined）；④ / 与 // 混淆（3.5 vs 3）。

1.7 章末作业

1. 写一张“自我介绍卡”：用变量存名字、年龄、爱好，再用 f-string 打印成一句话。
2. 两位数倒序：变量 num = 37，用 // 和 % 拆出十位和个位，打印成 73。
3. 验证：type(3.14) 是什么？len("linux") 是几？
4. 试错：故意写 print("5" + 5)，把报错抄进学习日志（这是你第一份“报错收藏”）。

 **对应练习册：基础篇阶段 1（题 1-1~1-8）**

1.8 这章你会了吗？

- ☐ 能背出 / 和 // 的区别（真除法 vs 整除）
- ☐ 会用 f-string 把变量塞进文字
- ☐ 知道 type() 怎么用、int/float/str 各是什么
- ☐ 能说出变量命名的三个规矩

📖 相关手册：print/input/type → 《Python 内置函数手册》第 1-2 组；f-string → 第 2 组尾部。

第 2 章 · 控制流: if、for、while

本章目标（学完能做什么）：会写成绩分段程序、九九乘法表、猜数字游戏——程序从此会"自己拿主意"和"重复干活"。

本章新知识点：`if/elif/else`（如果/否则如果/否则）· 比较运算符 · `and/or/not` · `range`（范围）· `for` · `while` · `break/continue` · `input`（读键盘）。

2.1 缩进是 Python 的"括号"

Python 用**缩进**（行首空格）表示代码属于哪个块，不像 Shell 用 `fi/done` 结尾。冒号：结尾的行下面，缩进 4 格的就是"块内内容"。

示范 1：最简单的 if

```
score = 85
if score >= 60:
    print("及格了")
print("程序结束")
```

```
及格了
程序结束
```

`if`（如果）后面跟条件，冒号结尾。缩进 4 格的 `print` 属于 `if` 块；不缩进的 `print` 不属于 `if`，无论如何都执行。缩进不一致 = 语法错误（后面 7 章详讲 `IndentationError`）。

示范 2：if / elif / else 三段论

```
score = 73
if score >= 90:
    print("优秀")
elif score >= 60:
    print("及格")
else:
    print("不及格")
```

```
及格
```

`elif` = 否则如果（再给一次机会），`else` = 否则（兜底）。从上往下**依次判断**，谁先满足就执行谁，后面都不看。所以 73 落在"大于等于 60"这档。

示范 3: 比较运算符 + 逻辑连接词

```
age = 19
print(age >= 18 and age < 60)
print(not (age < 18))
```

```
True
True
```

比较运算符: > < >= <= == (等于, 两个等号!) != (不等于)。and = 并且 (都真才真)、or = 或者 (有真就真)、not = 取反。结果叫**布尔值**: True (真) / False (假)。

2.2 for 循环: 把重复的事交给电脑

for (对于) 循环: 对一串东西, 一个一个拿出来处理。range (范围) 负责生成一串数字。

示范 1: range 生成数列

```
print(list(range(5)))
print(list(range(1, 10, 2)))
```

```
[0, 1, 2, 3, 4]
[1, 3, 5, 7, 9]
```

range(5) = 0 到 4 (共 5 个, 从 0 数); range(1, 10, 2) = 从 1 开始、到 10 为止**不含 10**、步长 2。
list() (转列表) 只是让它显示出来。规律: **左闭右开**——头有尾没有。

示范 2: for 遍历

```
for i in range(3):
    print("第", i, "次")
```

```
第 0 次
第 1 次
第 2 次
```

for 变量 in 一串东西: ——每次从"一串东西"里取一个放进变量 i, 缩进块执行一次。i 从 0 开始是 Python 惯例 (也是运维的常识: 编号从 0 数)。

示范 3: 嵌套 for (for 里再套一个 for)

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, j)
```

```
1 1  
1 2  
1 3  
2 1  
2 2  
2 3  
3 1  
3 2  
3 3
```

嵌套 for = 外层每转 1 圈，内层完整转完一轮。外层 i 控制"大步骤"、内层 j 控制"小步骤"——第 2 章作业的九九乘法表就是这个结构：外层 1~9 行，内层每行 1~i 列。

2.3 while 循环：满足条件就一直转

while (当.....时)：只要条件为真，就重复执行块。适合"不知道要转几次"的场景（比如猜数字，猜对为止）。

示范 1: 计数器

```
count = 0  
while count < 3:  
    print(count)  
    count = count + 1
```

```
0  
1  
2
```

三个零件：**初始值** (count=0)、**条件** (count<3)、**递增** (count = count + 1)。忘写递增 → 条件永远为真 → **死循环**（下面警告栏）。

⚠ **危险栏：死循环**。忘了写递增，while 会转到天荒地老。症状：终端一直刷字不停。急救：按 **Ctrl + C** 中断（和 Linux 一样）。写 while 前先在脑子里过一遍"这个循环怎么停下来"。

示范 2: break 与 continue

```
for i in range(1, 10):
    if i == 7:
        break
    if i % 2 == 0:
        continue
    print(i)
```

```
1
3
5
```

break (跳出) = 整个循环到此为止 (到 7 就停, 所以 6 之后的都不打印) ; **continue** (跳过本轮) = 这次不往下走, 直接下一轮 (偶数被跳过, 所以只打印奇数 1、3、5) 。

2.4 input(): 让程序问你话

示范: 读入并转换类型

```
age = input("你几岁: ")
print(type(age))
age = int(age)
print(type(age))
```

```
你几岁: 19
<class 'str'>
<class 'int'>
```

input() (读键盘) 返回的**永远是字符串**, 哪怕你敲的是 19。要算数必须先 **int()** (转整数) ——这是新手第一大坑: input 进来的东西不能直接做加减。**交互式示范**: 运行后程序会停住等你输入, 敲 19 回车 (上面"你几岁: 19"就是输入样例) ; 不输入直接跑会一直等着, 不会报错。

2.5 易错点 (这章的坑)

① while 忘递增 → 死循环 (Ctrl+C 急救) ; ② input 拿到的没 int() 转, 比大小出错; ③ 判断相等写成 = 而不是 == (= 是赋值不是比较) ; ④ 缩进不一致报 IndentationError。

本章对比: for vs while

对象	特点 / 适用场景
for i in range(n)	知道次数 (固定遍数) 时用; range 负责"数数"
while 条件	不知道次数 (直到满足条件) 时用; 要自己管计数器, 忘了递增会死循环

2.6 章末作业

1. 成绩分段程序: input 读入分数 (记得 int()) , 按 90/60 分档打印"优秀/及格/不及格"。
2. 九九乘法表: 嵌套 for (for 里再套一个 for) , 打印 1×1 到 9×9。
3. 猜数字游戏: 预设一个数字, while 循环 + input + if 判断, 猜对 break; 统计猜了几次。
4. 打印 1 到 100 里所有能被 7 整除的数 (提示: % 取余判断) 。

 对应练习册: 基础篇阶段 2 (题 2-1~2-8)

2.7 这章你会了吗?

- ☐ 能说出 if/elif/else 的执行顺序 (从上到下, 谁先满足执行谁)
- ☐ 能说出 for 和 while 的选择: 知道次数用 for, 不知道次数用 while
- ☐ 能说出 break 和 continue 的区别 (整锅端走 vs 跳过这一口)
- ☐ 知道 input() 返回的一定是字符串

 相关手册: range/enumerate → 《Python 内置函数手册》第 4 组; int/str 转换 → 第 2 组。

第 3 章 · 数据结构：list、dict、tuple、set

本章目标（学完能做什么）：会写通讯录小程序（能存、能查、能删）。数据结构是“数据处理”的核心地基——后面处理日志、JSON、API 全靠它们。

本章新知识点：list（列表）· dict（字典）· tuple（元组）· set（集合）· 索引/切片 · append · get · items · in 判断。

3.1 list 列表：一排盒子

比喻：列表 = 一列火车，每节车厢一个数据，车厢从 0 开始编号。

示范 1：创建、索引、负数索引

```
names = ["nginx", "docker", "mysql"]
print(names[0])
print(names[-1])
```

```
nginx
mysql
```

方括号 [] 创建列表。索引从 0 数：names[0] 是第一辆车厢。索引填**负数** = 从尾巴倒数：names[-1] 是最后一个。-1 永远取最后一项，非常常用。

示范 2：切片——取一段

```
ports = [80, 443, 22, 3306, 8080]
print(ports[1:3])
print(ports[:2])
print(ports[2:])
```

```
[443, 22]
[80, 443]
[22, 3306, 8080]
```

切片 [a:b] = 从编号 a 到编号 b**不含 b**（还是“左闭右开”）。省略开头 = 从头开始；省略结尾 = 到末尾。和 Linux 的 head -n（取头）/ tail -n（取尾）一个思路——ports[:2] 就是 head -2。

示范 3: append 追加、len、in 判断

```
services = ["nginx"]
services.append("docker") # append = 在末尾加一节车厢
print(services)
print(len(services))
print("docker" in services)
```

```
['nginx', 'docker']
2
True
```

`append()`（追加）是列表最常用的操作——想一次加一整串用 `extend`: `services.extend(["a", "b"])` 相当于连着 `append` 两次。区别一句话：**extend 把列表拆开逐个加，append 把整个列表当“一格”塞进去**（用 `print` 试一次就明白）。注意它是“点号调用”（方法，和函数写法不同，详见《Python 内置函数手册》理念篇）。`in` 判断某样东西在不在里面，返回 `True/False`——运维里“检查服务在不在清单里”天天用。

3.2 dict 字典：键值对

比喻：字典 = 通讯录。按“名字（键）”找“电话（值）”，而不是按位置找。和 Linux 的配置文件（键=值）完全一个思路。

示范 1: 创建与增删改查

```
server = {"ip": "10.0.0.5", "port": 22}
print(server["ip"])
server["owner"] = "运维组" # 加新键值对
server["port"] = 2222      # 改
print(server)
```

```
10.0.0.5
{'ip': '10.0.0.5', 'port': 2222, 'owner': '运维组'}
```

花括号 `{}` 创建。查用 `[键]`，加新键直接赋值，改也是赋值（键已存在就覆盖）。字典的“键”必须是不变类型（`str`、`int` 可以，`list` 不行——因为键要能唯一锁定位置）。

示范 2: get 安全取值 + items 遍历

```
server = {"ip": "10.0.0.5", "port": 22} # 独立运行请保留这行（不接上一示范）
print(server.get("ip"))
print(server.get("passwd", "没有这个键"))
for k, v in server.items():
    print(k, "=", v)
```

```
10.0.0.5
没有这个键
ip = 10.0.0.5
port = 22
```

`get()`（安全取值）和 `[键]` 的区别：键不存在时 `[键]` 直接报 `KeyError`，`get` 返回你指定的默认值（不给默认值就返回 `None`）。读配置文件的黄金习惯：**用 `get` 不用中括号**。`items()` 遍历时一次拿出“键, 值”两个，配 `for` 两个变量接。本例会转**两圈**：第一圈 `k="ip"、v="10.0.0.5"` → 打印一行；第二圈 `k="port"、v="22"` → 再打印一行——输出的两行就是这么来的。

3.3 tuple 元组与 set 集合

示范 1: tuple 不可变

```
point = (30, 120) # 圆括号创建
print(point[0])
point[0] = 40
```

```
30
Traceback (most recent call last):
  File "你的脚本名.py", line 3, in <module>
    point[0] = 40
TypeError: 'tuple' object does not support item assignment
（实际运行时上面是完整多行 Traceback，这里截取关键几行；报错读法见第 7 章）
```

tuple（元组） = 创建后**不能改**的列表。什么时候用：这组数据就是固定的（如坐标、日期），不想被误改。列表 vs 元组的选择：需要增删改 → `list`；固定不变 → `tuple`。

示范 2: set 去重

```
ips = ["10.0.0.1", "10.0.0.2", "10.0.0.1", "10.0.0.1"]
print(set(ips))
```

```
{'10.0.0.1', '10.0.0.2'} （set 不保顺序：这两项谁先谁后每次运行都可能不同）
```

set（集合） = 不存重复项、不保顺序的“去重机”。日志分析里“这堆 IP 里有哪些不同 IP”一句搞定。

3.4 易错点（这章的坑）

① 索引从 0 数（`names[1]` 是第二个）；② dict 用 `[键]` 取不存在的键报 `KeyError`——用 `get`；③ 切片 `[a:b]` 不含 `b`；④ `list` 能改、`tuple` 不能改，用错场合。

本章对比：四种容器

对象	特点 / 适用场景
list 列表 []	有序、可改、可重复——增删改查的主力
dict 字典 {键:值}	按键查值（像查字典）——一对对的数据
tuple 元组 ()	定死不可改——固定搭配的数据（坐标、日期）
set 集合 { }	不重复、没顺序——去重

3.5 章末作业

1. 通讯录小程序：一个 dict 存 3 个联系人（名字→电话）；实现查（get）、加（赋值）、删（后面 3.5 提示）。
2. 用 set 给名单去重：一个含重复名字的 list，打印去重结果和"名单原本几人、实际几人"。
3. 用切片从列表 [10, 20, 30, 40, 50, 60, 70] 取出 [30, 40, 50]（提示：中间一段）。
4. 写一个"服务端口表"：dict 存 nginx→80、ssh→22、mysql→3306，用 for + items() 逐行打印"服务名 监听 端口号"。

提示：删除字典键用 `del 字典[键]`（del = delete 删除）。

 **对应练习册：基础篇阶段 3（题 3-1~3-8）**

3.6 这章你会了吗？

- ☐ 能说出索引从 0 数、负数索引 -1 是什么
- ☐ 能说出切片 [a:b] 的"左闭右开"
- ☐ 能说出 dict 用 get 不用中括号的理由（不报 KeyError）
- ☐ 能说出 list/tuple 的选择标准（改不改）

📖 相关手册：list/dict/set/len → 《Python 内置函数手册》第 2、3 组；切片 → 第 2 组尾部。

第 4 章 · 函数：def、参数、返回值

本章目标（学完能做什么）：写自己的函数（问候、计算、判断）+ 一个“计算器”函数集合——代码从此可以打包复用。

本章新知识点：def（定义函数）· 参数 · return（返回）· print vs return 的区别 · 局部变量与作用域。

4.1 函数 = 把重复操作打包成一条“新命令”

比喻：函数就像 Linux 里的 alias——把一串长操作打包成一条短命令，以后敲名字就行。区别是函数更强大：能接受不同输入、能返回结果。

示范 1：def 定义 + 调用

```
def greet(name):  
    print(f"你好, {name}!")  
  
greet("小明")  
greet("运维组")
```

```
你好, 小明!  
你好, 运维组!
```

def（定义）后面跟函数名和括号里的**参数**（原料入口），冒号结尾，缩进块是函数体。定义时只写一次，调用时传不同参数就出不同结果——同一台机器，换原料。

示范 2：return 返回结果

```
def add(a, b):  
    return a + b  
  
result = add(3, 5)  
print(result)  
print(add(10, 20) * 2)
```

```
8  
60
```

return（返回）= 把计算结果**交出去**。交出去之后，调用处可以存变量、可以继续运算（add(10,20)*2 就是先算 30 再乘 2）。

示范 3: print 与 return 的区别 (新手第一迷思)

```
def f1(x):  
    print(x * 2)    # 只是打印, 没把结果交出去  
  
def f2(x):  
    return x * 2    # 把结果交出去  
  
a = f1(5)  
b = f2(5)  
print("a =", a)  
print("b =", b)
```

```
10  
a = None  
b = 10
```

f1 里的 print 把 10 显示在屏幕上, 但没把结果交给变量 a, 所以 a 是 None (空)。f2 的 return 把 10 交给 b, 所以 b = 10。print 是"展示给人看", return 是"交给程序用"。需要继续算/存, 必须 return。

4.2 参数与作用域

示范 1: 默认参数 + 关键字参数

```
def check_disk(threshold=80, mount="/"):  
    print(f"检查{mount}, 阈值{threshold}%")  
  
check_disk()  
check_disk(mount="/data", threshold=90)
```

```
检查/, 阈值80%  
检查/data, 阈值90%
```

参数给默认值后, 调用时可以不传。指定参数名传 (mount=...) 叫**关键字参数**, 顺序随便换——运维脚本里这样写, 别人读代码一眼就懂。

示范 2: 多返回值 (其实是元组)

```
def divmod2(a, b):  
    return a // b, a % b  
  
shang, yu = divmod2(17, 5)  
print(shang, yu)
```

```
3 2
```

return 后面逗号隔开多个值 = 一次交回一个元组, 用多个变量拆开接。"一次算两样、拆开存"是运维函数里的常见套路。

作用域（一句话版）：函数内部的变量是**局部变量**，函数外面看不到；函数外面的变量函数里可以读，但不能直接改（要改得用 `global` 声明——日常几乎不用，知道即可）。变量命名的“盒子”只在定义它的区域有效。

4.3 易错点（这章的坑）

① 忘写 `return`，函数返回 `None`；② 定义了函数但没调用（`def` 只是“登记”，不调用不执行）；③ 想在函数里改外面的变量（作用域限制）；④ 把 `print` 当 `return` 用。

4.4 章末作业

1. 写三个小函数体会“输入→加工→输出”三要素：`double(x)`（返回两倍）、`greet(name)`（返回问候语）、`is_even(n)`（返回布尔值）。
2. 写一个“计算器”函数集合：加、减、乘、除四个函数（`div` 函数记得除数为 0 时返回提示）。
3. 写一个 `check_usage(percent)` 函数：传入磁盘使用率，`>= 90` 返回“红色告警”，`>= 80` 返回“黄色提醒”，否则返回“正常”（提示：`return` 字符串）。
4. 验证 `print` vs `return`：写一个“有 `return`”和“只有 `print`”的函数，各自赋值给变量后 `type()` 看看是什么。

 对应练习册：基础篇阶段 4（题 4-1~4-4、4-7、4-8）

4.5 这章你会了吗？

- ☐ 能说出 `def` 的完整格式（函数名 + 参数 + 冒号 + 缩进块）
- ☐ 能讲清 `print` 与 `return` 的区别（展示 vs 交出）
- ☐ 会用默认参数和关键字参数
- ☐ 能说出什么时候该把代码封装成函数（重复两次以上就封）

📖 相关手册：`print/return` → 《Python 内置函数手册》第 1 组；自定义函数速查 → 第 6 组。

第 5 章 · 模块与导入：import、os、time

本章目标（学完能做什么）：写“我的工作目录报告”——列出当前目录所有文件和修改时间。你开始调用 Python 自带的大工具箱（标准库）。

本章新知识点：`import`（导入）· 模块与标准库 · `os` 模块 · `time` 模块 · `sys.argv` 初步。

5.1 import 三种写法

比喻：模块 = 工具箱。import 就是“把工具箱搬过来”。标准库 = 装 Python 时免费附赠的一排工具箱，不用安装，直接用。

示范 1：三种导入写法

```
import os                # 写法1：整个模块搬进来，用 os.xxx 调用
print(os.getcwd())

from os import getcwd    # 写法2：只拿一件工具，直接用名字
print(getcwd())

import os as o           # 写法3：起个外号（as = 当作）
print(o.getcwd())
```

```
/home/你的真实用户名
/home/你的真实用户名
/home/你的真实用户名 （示例值：实际以你的当前目录为准，在哪个目录运行就是哪个）
```

三种写法结果一样。习惯：**写法 1 最推荐**（`os.getcwd` 一眼看出这工具来自哪，代码可读性好）。`getcwd()`（get current working directory 获取当前工作目录）就是 Linux 的 `pwd`。

5.2 os 模块：Python 版的 pwd/ls

示范 1：listdir 列目录 + path.join 拼路径

```
import os
files = os.listdir(".")
print(files)
print(os.path.join("lab", "notes.md"))
```

```
['hello.py', 'lab', 'notes.md'] （示例值：实际列的是你当前目录里的内容，机器不同结果不同）
lab/notes.md
```

`os.listdir()`（列目录）= Linux 的 `ls`；参数 `"."` 表示当前目录（和 Linux 一样）。`path.join`（拼路径）= 把几段用正确的斜杠连起来——永远用它拼，别手写斜杠（Linux 用 `/`、Windows 用 `\`，`join` 自动处理）。示例里 `"student"` 换成你的真实用户名（`whoami` 可查）。`os.path.join()`（拼接路径）自动处理斜杠——永远用它拼路径，别手写字符串相加。

示范 2: 读取文件大小和修改时间

```
import os, time
# 前置 (终端执行): echo 'hello world!' > hello.txt
path = "hello.txt"
if os.path.isfile(path):
    print(os.path.getsize(path))
    stamp = os.path.getmtime(path)
    print(time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(stamp)))
else:
    print("文件不存在")
```

12

2026-08-25 14:30:12 (示例时间; 实际值取决于文件)

(新目录没建文件时, 会走 else 分支打印"文件不存在"—两个分支都试一遍)

isfile 先确认是普通文件 (目录用 isdir 判断, 写法对称: os.path.isdir(路径) → True/False);
getsize 取字节数; getmtime 取修改时间戳; localtime 转成本地时间, 最后由 strftime 格式化。

示范 3: os.rename 重命名 / 移动文件 (批量改名工具的核心)

```
import os
# 前置 (终端执行): touch old.txt
os.rename("old.txt", "new.txt") # 改名: old.txt 变成 new.txt
print(os.path.exists("new.txt")) # 存在性检查: True
```

True

os.rename(旧名, 新名) = Linux 的 mv: **同目录=改名, 跨目录=移动**。注意它是**直接覆盖目标** (目标已存在会被替换) ——所以进阶篇场景 1 的批量改名工具会先做冲突检查再执行。改名前想"先看清楚", 用 os.path.exists (判断文件在不在) 探路。这是你未来第一个正式脚本 (批量文件重命名) 的核心 API。

5.3 time 模块: 时间戳、暂停、格式化

示范 1: time() 与 strftime

```
import time
print(time.time())
print(time.strftime("%Y-%m-%d %H:%M:%S"))
```

1787612345.678

2026-08-25 14:30:12 (示例值: 时间戳每次运行都不同, 日期是运行当天, 不要拿它当正确答案)

time.time() (时间戳) = 从 1970-01-01 起的秒数, 给机器算时间差用 (两个相减 = 间隔秒数)。
strftime() (格式化时间) = 把时间变成人读的格式: %Y 年、%m 月、%d 日、%H 时、%M 分、%S 秒。

示范 2: sleep 暂停 + sys.argv 接命令行参数

```
# 启动命令（终端执行）：python3 你的脚本名.py 参数1 参数2
import time, sys
print("开始")
time.sleep(2)
print("两秒后")
print(sys.argv)
```

```
开始
两秒后
['你的脚本名.py', '参数1', '参数2']
```

`sleep()`（暂停）= 程序歇几秒（轮询监控时控制节奏用）。`sys.argv` = 命令行传进来的参数列表：
`argv[0]` 是你实际保存的脚本文件名（不是固定的 `app.py`，存成什么名就是什么），`argv[1]` 起才是真正的参数——第 9 章 `argparse` 的底层就是它。

5.4 random 模块：掷骰子

示范 4：随机数

```
import random
random.randint(1, 6)          # 随机给一个 1~6 的整数（像掷骰子）
print(random.randint(1, 100)) # 每次运行结果都不一样
```

42 （每次运行都会变，以你机器的实际输出为准）

`random.randint(a, b)` = 随机返回 `a` 到 `b` 之间的整数（**两头都包含**）。要先 `import random` 才能用。猜数字、抽奖、随机点名这类小游戏都靠它。

5.5 易错点（这章的坑）

① 忘 `import` 就调用（`NameError: name 'os' is not defined`）；② 手拼路径字符串（用 `os.path.join`）；③ `strftime` 格式串写错（`%H` 时 / `%M` 分搞反）；④ `import` 写在函数里（习惯统一放文件顶部）。

5.6 章末作业

1. “我的工作目录报告”：先 `import os`，用 `os.listdir(".")` 列出当前目录；对普通文件用 `os.path.getsize(路径)` 取得字节数并打印文件名和大小。
2. 对普通文件用 `os.path.getmtime(路径)` 取得修改时间戳，再用 `time.strftime` 与 `time.localtime` 格式化后打印。
3. 用 `import time` 和 `time.sleep` 写一个“倒计时 3 秒”程序（3、2、1、出发）。
4. 用 `import sys`，打印 `sys.argv`：运行 `python3 脚本名 a b c`，观察 `argv` 里有什么。

 对应练习册：基础篇阶段 4（题 4-5、4-6）

5.7 这章你会了吗？

- ☐ 能说出 import 三种写法，并说清推荐哪种、为什么
- ☐ 能用 os 模块做 pwd (getcwd) 和 ls (listdir)
- ☐ 能说出 strftime 里 %Y %m %d %H %M %S 各是什么
- ☐ 知道 sys.argv[0] 和 argv[1] 分别是什么

📖 相关手册：os/time 常用函数 → 《Python 内置函数手册》第 6 组。

第 6 章 · 文件操作：open/with、json

本章目标（学完能做什么）：把通讯录存成 JSON 文件再读回来——程序从此能**记住数据**（关掉再开不丢失）。

本章新知识点：open（打开文件）· r/w/a 三种模式 · with（自动关闭）· 逐行读 · json 模块。

6.1 open 的三种模式：r 读、w 写、a 追加

呼应 Linux：w 就像 >（覆盖重定向），a 就像 >>（追加重定向），r 就像 cat。你在 Linux 里学过的“覆盖 vs 追加”在这里一模一样。

示范 1：w 写与 r 读

```
f = open("test.txt", "w", encoding="utf-8")
f.write("第一行\n第二行\n")
f.close()
```

```
f = open("test.txt", "r", encoding="utf-8")
print(f.read())
f.close()
```

```
第一行
第二行
```

open()（打开文件）第一个参数是文件名，第二个是模式。“w”写、“r”读、“a”追加。encoding="utf-8" 保证中文不乱码（永远带上）。close()（关闭）用完必须关——就像 Linux 里用完退出。

⚠ 危险栏：w 模式会清空原文件。对已存在的文件用“w”打开，**内容立刻被清空**，再写新内容。重要文件永远用“a”或先备份——和 Linux 里 echo x > file 覆盖文件是一个道理。

示范 2：a 追加 + with 自动关闭

```
# 前置：承接示范 1（test.txt 已含前两行）；也可以先用 w 模式初始化一遍：with
open("test.txt", "w") as f: f.write("第一行\n第二行\n")
```

```
with open("test.txt", "a", encoding="utf-8") as f:
    f.write("第三行\n")
```

```
with open("test.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

```
第一行
第二行
第三行
```

“a”追加：内容接在尾巴，不丢旧的。**重复运行会继续追加**（再跑一次就多一行，行数 = 初始两行 + 运行次数），这不是 bug——想要干净输出就先用 w 模式重新初始化。with ... as（带上下文打开）= 缩进块结束**自动关闭文件**，不用手写 close，忘关也不怕。黄金习惯：**永远用 with open**。

示范 3：逐行读文件（处理日志的起手式）

```
# 前置（终端执行，可复制）：mkdir -p ~/lab && cd ~/lab && echo 第一行 > test.txt && echo
第二行 >> test.txt && echo 第三行 >> test.txt
with open("test.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

```
第一行
第二行
第三行    （行数与你的 test.txt 内容对应）
```

先认识"对象"："对象"就是把**数据 + 操作打包在一起的盒子**——`open()` 返回的"文件对象"自带 `.read()/.close()` 等操作，点号 (.) 调用就是让盒子干活。文件对象可以直接 `for` 遍历（一行一行拿）。`strip()`（去空白）去掉每行末尾的换行符，否则会多打空行。这个"逐行读文件"就是日后分析日志的起手式。

6.2 json 模块：数据存成标准格式

比喻：json = 世界通用的"数据信封"格式。Python 的 dict 和它几乎一样（键值对），所以转换极顺。

示范 1：dumps/loads 与 dump/load 两对

```
import json
data = {"name": "小明", "age": 19}

text = json.dumps(data, ensure_ascii=False) # dict → 字符串
print(text)

back = json.loads(text) # 字符串 → dict
print(back["name"])
```

```
{"name": "小明", "age": 19}
小明
```

带 s 的管字符串：dumps = dict 转字符串，loads = 字符串转 dict。**不带 s 的管文件：**dump(数据, 文件) 写进文件，load(文件) 从文件读出。口诀：**带 s 是字符串 (string)，不带 s 是文件。**

示范 2: dump 到文件再 load 回来

```
import json
contacts = {"小明": "13800000000", "老王": "13900000000"}

with open("contacts.json", "w", encoding="utf-8") as f:
    json.dump(contacts, f, ensure_ascii=False)

with open("contacts.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)

print(loaded.get("小明"))
```

```
13800000000
```

dict 存成 contacts.json 文件（cat 看看：就是可读的键值对文本），再读回来还是 dict。这就是“程序记住数据”的标准姿势——通讯录、配置、API 返回全是这套。

6.3 易错点（这章的坑）

① "w" 打开已有文件 → 内容立刻清空；② 忘 encoding="utf-8" → 中文乱码；③ 忘 close → 用 with open 根治；④ 读的文件路径写错 → FileNotFoundError。

6.4 章末作业

1. 把第 3 章通讯录用 json.dump 存成 contacts.json，再用 json.load 读回来打印。
2. 统计一个文本文件的行数和字数（提示：逐行读，行数 += 1，字数 += len(line.strip())）。
3. 对比实验：对同一个文件分别用 "w" 和 "a" 写入一句话，观察哪种会清空原内容（用临时文件做）。
4. 写一个“给文件加日期前缀”的小程序：读当前目录某文件，复制成 "2026-08-25_原文件名"（提示：os.path.join + 读内容写内容）。

 对应练习册：基础篇阶段 5（题 5-1~5-4）

6.5 这章你会了吗？

- ☐ 能说出 r/w/a 三模式，w 为什么危险
- ☐ 永远用 with open，能说出为什么（自动关闭）
- ☐ 能背出 json 口诀（带 s 字符串，不带 s 文件）
- ☐ 会逐行读文件并 strip 换行符

📖 相关手册：open/read/write → 《Python 内置函数手册》“文件对象三连”；JSON 与字符串方法 → 对应速查小节。

第 7 章 · 异常处理：try/except/finally

本章目标（学完能做什么）：给第 6 章的 JSON 程序加上“文件不存在时友好提示”——你的脚本从此**不会一碰错误就崩溃**。学完本章 = 达成检查点②：能独立写带 try/except 的 Python 脚本。

本章新知识点：Traceback 怎么读 · try/except（尝试/接住）· 捕获指定异常 · finally（无论如何都执行）· 常见异常对照表。

7.1 程序为什么报错 + Traceback 怎么读

呼应 Linux：报错处理 = Linux 里的“看日志找根因”。你背过的报错英语（《报错英语单词手册》），现在直接上场。

示范 1：完整 Traceback 逐行拆解

```
open("不存在的文件.txt")
```

```
Traceback (most recent call last):
  File "app.py", line 1, in <module>
    open("不存在的文件.txt")
FileNotFoundError: [Errno 2] No such file or directory: '不存在的文件.txt'
```

从下往上看（和读日志一样）：① 最后一行 = 错误本质：FileNotFoundError（文件找不到）+ 哪个文件；② 往上一段 = 凶手代码 open(...); ③ 再往上 = 案发地点 File "app.py", line 1（第 1 行）。先把最后一行读明白，80% 的报错就解决了。详细读法见《日志报错解读手册》B 组。

7.2 try/except：尝试，出错就接住

示范 1：捕获指定异常

```
import json # 每个示范都能独立复制运行
try:
    with open("contacts.json", "r", encoding="utf-8") as f:
        data = json.load(f)
except FileNotFoundError:
    print("通讯录文件还没创建，先用空通讯录。")
    data = {}
print("继续干活：", data)
```

```
通讯录文件还没创建，先用空通讯录。
继续干活： {}
```

try（尝试）块里放“可能出错”的代码；**except**（接住）指定异常类型——只有 FileNotFoundError 才被接住，其他错误照常报。接住后程序**不会崩溃**，继续往下走。指定类型捕获是黄金习惯（见下方橙色提示框，讲为什么）。

示范 2: finally 无论如何都执行

```
try:
    print("干活中...")
    x = 1 / 0
except ZeroDivisionError:
    print("除数为零了!")
finally:
    print("收尾工作: 关文件、发通知 (这句总会执行)")
```

干活中...
除数为零了!
收尾工作: 关文件、发通知 (这句总会执行)

finally (最后) 块不管出不出错都执行——适合放"必须收尾"的动作 (关文件、关连接)。 with open 已经自动关了文件, 所以日常 finally 用得少, 知道它存在即可。

坑: 不要无脑 try。 有的错误就该让它报出来 (比如逻辑写错), 接住了反而掩盖问题。**原则: 只有"预期内可能发生的"错误 (文件不存在、网络断开、用户乱输入) 才 try;** 代码 bug 不要 try, 让它报错、修掉它。

7.3 常见异常对照表

异常	什么时候发生
FileNotFoundError	文件不存在
PermissionError	没权限 (对应 Linux Permission denied)
KeyError	字典键不存在 (用 [键] 取)
IndexError	列表索引超出范围
ValueError	类型对但值不对 (int("abc"))
TypeError	类型不对 ("5" + 5)
ZeroDivisionError	除数为零
KeyboardInterrupt	Ctrl+C 打断 (一般不接住)

完整版 (带逐词拆解和解决办法) → 见《日志报错解读手册》B 组, 遇到报错当字典查。

7.4 易错点 (这章的坑)

① 无脑 try 掩盖代码 bug; ② except 后面不写异常类型, 什么错都接住, 查错困难; ③ try 块太大 (只包住"可能出错的那一句"); ④ except 里既不打印也不记录, 错误被吞掉。

7.5 章末作业

1. 给第 6 章 JSON 程序加异常处理: 文件不存在时打印友好提示并用空 dict 继续。
2. 写一个"安全除法"函数: 除数为 0 时返回提示文字而不是报错。
3. 故意触发三种不同异常 (FileNotFoundError、KeyError、ValueError), 抄下三种 Traceback 的最后一行, 对照 7.3 表说出各是什么错。
4. 把 1 的脚本跑两遍: 第一次无文件 (走 except), 第二次有文件 (走 try), 验证两条分支都正常。

7.6 这章你会了吗？

- ☐ 会从下往上读 Traceback，先说最后一行是什么
- ☐ 能说出 try/except 的写法，会捕获指定异常
- ☐ 能说出 finally 的作用（无论如何都执行）
- ☐ 能说出"不要无脑 try"的原则

 相关手册：Traceback 逐行拆解 → 《日志报错解读手册》B 组开篇；报错英语 → 《报错英语单词手册》。

第 8 章 · 网络请求：requests

本章目标（学完能做什么）：写“天气查询器”雏形——程序联网拿数据。这是脚本 2 的核心能力，也是调 API 的第一步。

本章新知识点：HTTP 请求/响应 · 状态码 · `requests` 模块 · 响应对象 · 超时与异常。

📌 外部扩展声明：`requests` 是**第三方库**（需先安装），本章所有示范都调**外部 API 并需要联网**。离线/未安装时：安装方式已标注，无法联网时用离线练习（读一段保存好的 JSON 文本跑 `.json()`/取值逻辑），主线能力是“拿响应→解析→异常处理”，不在网络本身。练习册基础篇 6-1/6-2 已标外部扩展。

8.1 什么是 HTTP 请求：Python 版的 curl

呼应 Linux：`requests.get()` 就是 `curl`——向网址发一个“请给我数据”的请求，对方回一个响应。你在第 8 章 Linux 课里用过的 `curl`，现在换成 Python 写。

API 与响应对象：API（Application Programming Interface，应用程序接口）就是“程序之间说话的接口”——别人把服务做成网址，你按约定请求、它按约定回数据，“调 API”就是这个意思。服务器回的东西存进变量 `r`，`r` 叫**响应对象**：把状态码、内容、响应头打包在一个盒子里（`r.status_code`、`r.json()` 都是从盒子里取东西）。

示范 1：发请求 + 看状态码

```
import requests # 需要先安装: sudo apt install python3-requests
r = requests.get("https://api.open-meteo.com/v1/forecast?
latitude=30.25&longitude=120.17&current_weather=true", timeout=10)
print(r.status_code)
```

200

`requests` 是第三方库，Ubuntu 要先装：`sudo apt install python3-requests`（平板 Termux 用户：`pkg install python-requests`）。`get()`（发送 GET 请求）返回**响应对象** `r`。`status_code`（状态码）：**200 = 成功**、**404 = 不存在**、**500 = 对方服务器出错**。`timeout`（超时）= 等多久没回应就算失败（10 秒），必须写——不然对方没响应你的程序会永远卡住。

示范 2：拿数据 `.json()` 与 `.text`

```
import requests
r = requests.get("https://api.open-meteo.com/v1/forecast?
latitude=30.25&longitude=120.17&current_weather=true", timeout=10)
data = r.json()
print(data["current_weather"]["temperature"])
```

31.2 （示例—实际温度随请求时刻变化）

`json()`（转字典）= 把响应内容解析成 `dict`——第 6 章 `json.loads` 的姊妹，这次直接从网上来。然后就是熟悉的 `dict` 取值：`data["current_weather"]["temperature"]` 层层拿。返回的是 `dict`，你就会处理它（第 3 章学的）。

8.2 网络失败怎么办：try/except 接住

示范 1：断网/超时也不崩

```
import requests
try:
    r = requests.get("http://127.0.0.1:9", timeout=3) # 本机 9 端口没有服务，连接必然被拒
    # 绝—受控失败，不依赖外网
    print(r.status_code)
except requests.exceptions.RequestException:
    print("网络出问题了：断网？超时？服务器没响应？")
```

网络出问题了：断网？超时？服务器没响应？

第 7 章的 try/except 直接上场（`except 类型 as 变量` 可把错误对象接住，如 `except Exception as e: print(e)`——第 7 章作业会用到）：RequestException 是 requests 所有网络错误的总称（断网、超时、域名解析失败都归它管）。调 API 的脚本必须套 try/except——网络永远可能出问题，这是运维的常识。想同时验证成功分支：把 127.0.0.1:9 换成 8.1 用过的真实 API 地址再跑，就会走成功分支打印 200——同一段代码两个分支都试一遍。

8.3 易错点（这章的坑）

- ① 忘装 requests (ModuleNotFoundError) ； ② 忘写 timeout (对方无响应程序卡死) ； ③ URL 拼错 (404 或连不上) ； ④ 不套 try/except, 断网就崩。

本章对比：requests vs curl

对象	特点 / 适用场景
curl (Linux 命令)	终端里随手测一下接口；快速、无代码
requests (Python 库)	脚本里拿数据做后续处理；json() 直接转 dict，配合 if/try 做逻辑

8.4 章末作业

1. 请求 Open-Meteo 接口，打印当前温度（网址就用示范 1 里的）。
2. 写天气查询器雏形：把城市经纬度做成两个变量，打印“城市 当前温度 X°C”。
3. 故意请求一个不存在的地址（如 `api.open-meteo.com/v1/forecast?latitude=abc`），看状态码或报错是什么。
4. 给 1 的程序加 try/except 和 timeout，然后断网跑一遍验证提示正常。

 对应练习册：基础篇阶段 6（题 6-1、6-2）；进阶场景 2（天气查询器）

8.5 这章你会了吗？

- ☐ 能说出 200/404/500 状态码的含义
- ☐ 会用 json() 把响应转成 dict 并取值
- ☐ 知道 timeout 必须写，以及为什么
- ☐ 会用 try/except 接住网络错误

📖 相关手册：json 处理 → 《Python 内置函数手册》；curl 对照 → 《Linux 入门》第 8 章。

第 9 章 · 命令行参数：argparse

本章目标（学完能做什么）：脚本支持 `python3 weather.py 杭州` 这种命令行调用——同一脚本应对不同情况，从“玩具”变“工具”。

本章新知识点：argparse 三步走 · 位置参数 · --可选参数 · 默认值与 help 文本。

9.1 为什么脚本需要参数

呼应 Linux：Linux 命令都有参数（`ls -l /home`）。好脚本也该这样：敲命令时顺手指明“对谁做、怎么做”，而不是改代码。argparse 就是给 Python 脚本装参数的工具。

示范 1：argparse 三步走（创建 → 添加 → 解析）

```
import argparse

parser = argparse.ArgumentParser(description="天气查询器") # 1 创建解析器
parser.add_argument("city", help="要查的城市")           # 2 添加参数
args = parser.parse_args()                                # 3 解析

print(f"正在查{args.city}的天气...")
```

```
$ python3 weather.py 杭州
正在查杭州的天气...
```

三步走：① **ArgumentParser()**（创建解析器）→ ② **add_argument()**（添加参数，"city" 不带 -- 就是位置参数）→ ③ **parse_args()**（解析命令行）。之后 `args.city` 就是传入的城市。description 会显示在自动生成的 --help 里。

示范 2: --可选参数 + 默认值 + 自动 --help

```
import argparse
parser = argparse.ArgumentParser(description="天气查询器")
parser.add_argument("city", help="要查的城市")
parser.add_argument("--days", type=int, default=1, help="查未来几天，默认 1")
args = parser.parse_args()
print(args.city, args.days)
```

```
# 第一次运行：传两个参数
$ python3 weather.py 杭州 --days 3
杭州 3

# 第二次独立运行：看自动生成的帮助
$ python3 weather.py --help
usage: weather.py [-h] [--days DAYS] city
```

天气查询器

positional arguments:

city	要查的城市
------	-------

options:

-h, --help	show this help message and exit
--days DAYS	查未来几天，默认 1

带 -- 的是**可选参数**：不传就用 default（默认值）。**--help 是自动生成的**——和 Linux 的 --help 同源（一个祖先），你早就熟了。type=int 让 argparse 自动把输入转成整数。

示范 3: 开关参数 action="store_true"（不传值，只回答"有/没有"）

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--dry-run", action="store_true", help="只演练不真改")
args = parser.parse_args()
print(args.dry_run)
```

```
# 不传 --dry-run: False（没这个开关）
$ python3 demo.py
False

# 传了 --dry-run: True（开关打开了）
$ python3 demo.py --dry-run
True
```

action="store_true" = 开关型参数：**传了就是 True，不传就是 False**，后面不跟值。适合 --dry-run（只演练）、--verbose（啰嗦输出）这种“要不要开某功能”的开关。注意：开关参数**不能**写 type=bool（那样"--dry-run false" 会得到字符串 "false"，反而出错）——store_true 才是正确姿势。

对比：argparse 和 sys.argv（第 5 章）的区别：argv 是毛坯（一系列字符串，类型、顺序全要自己处理）；argparse 是精装（自动类型转换、自动 --help、自动报“缺参数”）。**正式脚本一律用 argparse。**

9.2 易错点（这章的坑）

① 位置参数顺序写反（args 里按敲的顺序进）；② 忘 type=int（拿到的是字符串，没法比大小）；③ 解析后属性名写错（args.city 写成 args.citys）；④ 位置参数和可选参数混加（先加位置参数，再加 --可选）。

9.3 章末作业

1. 给第 4 章计算器加参数化：python3 calc.py 3 + 5 打印 8（提示：位置参数三个）。
2. 给通讯录加子命令：add / find / list 三种用法（提示：用第一个位置参数当“命令名”，if 分支处理）。
3. 写一个带 --threshold 可选参数的磁盘检查脚本（默认 80，传入则用传入值）。
4. 敲 python3 脚本名 --help，看 argparse 自动生成的帮助文本长什么样，抄进日志。

 **对应练习册：基础篇阶段 6（题 6-3、6-4）；进阶场景 1（批量重命名工具）**

9.4 这章你会了吗？

- ☐ 能背出 argparse 三步走（创建 → 添加 → 解析）
- ☐ 能说出位置参数和 --可选参数的区别
- ☐ 知道 --help 是自动生成的
- ☐ 知道 default 和 type 两个参数的用途

📖 相关手册：--help 对照 → 《Linux 入门》第 0 章。

第 10 章 · 日志：logging

本章目标（学完能做什么）：给脚本加日志记录——每次操作留痕，出问题能排查。日志就是运维的第一现场（你读过 Linux 日志，现在自己生产日志）。

本章新知识点：print 调试的不足 · logging 模块 · 四个级别 · basicConfig（基本配置） · 日志写文件。

📌 口径说明：logging 标准是**五级**（DEBUG/INFO/WARNING/ERROR/CRITICAL）；本章为入门只讲前四级（到 ERROR），属**简化范围**——真实项目里 CRITICAL 照样常用（定义见《日志报错解读手册》级别表）。

10.1 print 调试的不足

print 能看结果，但有三个毛病：① 没有**级别**（分不出“正常”和“出错了”）；② 没有**时间**（不知道何时发生）；③ 用完要删（上线时满屏 print 是灾难）。logging（日志）模块一次解决三样。

示范 1：四个级别

```
import logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

logging.debug("调试细节")
logging.info("服务启动")
logging.warning("磁盘快满了")
logging.error("连接失败")
```

```
2026-08-25 14:30:12,345 INFO 服务启动
2026-08-25 14:30:12,345 WARNING 磁盘快满了
2026-08-25 14:30:12,346 ERROR 连接失败
```

四个级别从低到高：DEBUG（调试细节）→ INFO（正常消息）→ WARNING（警告）→ ERROR（错误）。basicConfig（基本配置）里 level=INFO 的意思是“只显示 INFO 及以上”——所以 debug 没打印。**format 串**决定每行长什么样：时间 + 级别 + 消息，和你读过的 Linux 日志一个结构（时间+谁+说什么）。

示范 2：日志写进文件

```
import logging
logging.basicConfig(filename="app.log", level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
logging.info("程序启动")

# 之后终端里看：cat app.log
```

```
$ cat app.log
2026-08-25 14:31:00,001 INFO 程序启动
```

加 filename="app.log"，日志就写进文件而不是屏幕——和 Linux 的“日志落到 /var/log”一个思路：程序跑着，日志记着，出事翻日志。每个脚本都应该有自己的日志文件。

10.2 易错点 (这章的坑)

① level 设成 ERROR, INFO 日志全被过滤看不见; ② format 串写错 (%(asctime)s 漏写 s); ③ 上线脚本还在用 print 调试; ④ filename 用相对路径, 日志文件"不知去向"。

10.3 章末作业

1. 给第 9 章的磁盘检查脚本加日志: 每次检查记一条 INFO"检查完成, 使用率 X%", 超阈值记 WARNING。
2. 把日志写到文件, 跑完后 cat 看内容。
3. 实验 level: 分别把 level 设为 DEBUG 和 ERROR, 观察同一段代码打印出什么差别。
4. 把错误信息写进日志: try/except 里 except 块加 logging.error(f"失败原因: {e}")。

 对应练习册: 基础篇阶段 6 (题 6-5、6-6)

10.4 这章你会了吗?

- ☐ 能背出四个级别及含义 (DEBUG/INFO/WARNING/ERROR)
- ☐ 能说出 level=INFO 的作用 (屏蔽 DEBUG)
- ☐ 会用 filename 把日志写进文件
- ☐ 能说出 print 调试的三大不足

📖 相关手册: 日志报错解读 → 《日志报错解读手册》; /var/log 对照 → 《Linux 入门》第 10 章。

第 11 章 · 进程调用：subprocess

本章目标（学完能做什么）：用 Python 调 `df -h`、`free -h` 并处理结果——Linux 三连和 Python 三连在这里合流。本章是通往王牌项目 `server-health-check` 的接口。

本章新知识点：`subprocess.run`（运行命令）· `capture_output` · `text` · 返回码。

11.1 为什么 Python 要调系统命令

比喻：Python 是大脑，系统命令是双手。大脑想做“磁盘检查 + 超阈值告警”，但查磁盘这种活是 `df` 的专长——那就让大脑指挥双手：Python 调 `df` 拿数据，再判断、告警。这就是两条线合流。

示范 1：run 调 df 并拿输出

```
import subprocess
result = subprocess.run(["df", "-h", "/"], capture_output=True, text=True)
print(result.stdout)
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	916G	146G	724G	17%	/

（示例值：你的机器容量不同，数字不同属正常）

`run()`（运行）第一个参数是**命令和参数组成的列表**（注意：每段一个元素，不像终端里敲整串）。
`capture_output=True` = 把输出抓回来；`text=True` = 输出是字符串而不是字节。`result.stdout` 就是 `df` 的输出——Linux 里学的 `df -h`，现在变成了 Python 里的数据。

示范 2：判断返回码（成功还是失败）

```
import subprocess
ok = subprocess.run(["ls", "/home"], capture_output=True)
print(ok.returncode)
bad = subprocess.run(["ls", "/不存在"], capture_output=True)
print(bad.returncode)
```

0
2

`returncode`（返回码）= 命令的“成绩单”：**0 = 成功**，非 0 = 失败（数字是失败原因编号，和你 Linux 里 `echo $?` 看到的是一回事）。脚本里用 `returncode` 判断“命令成了没”，比看输出文字靠谱得多。

示范 3：拿 free -h 的输出做判断（王牌项目雏形）

```
import subprocess
r = subprocess.run(["free", "-h"], capture_output=True, text=True)
lines = r.stdout.split("\n")
print(lines[1])
```

Mem:	31G	2.1G	26G	120M	2.3G	28G
------	-----	------	-----	------	------	-----

组合拳：`subprocess` 拿输出 → `split()`（切割字符串，第 1 章 1.3 学过）按行分成列表 → 取第 2 行（`lines[1]`，第 1 行是表头）→ 再 `split` 按空格切 → 拿到数字。这套“调命令 + 解析输出”就是 `server-health-check` 的核心动作。到这一步，Linux 的知识全部变成了 Python 的数据来源。

⚠ **安全提醒：**用 subprocess 调命令时，命令列表里的参数**不要直接拼用户输入**（尤其是 rm、dd 这类危险命令）。永远用列表形式传参，别用字符串拼接——以后做 Web 服务时会懂，先种下这个习惯。

11.2 易错点（这章的坑）

① 命令写成字符串而不是列表（["df","-h"] 对，"df -h" 错）；② 忘 capture_output=True，拿不到输出；③ 忘 text=True，输出是字节串 b'...'；④ 忘判断 returncode，命令失败也不知道。

本章对比：subprocess vs 直接在 Shell 敲

对象	特点 / 适用场景
直接在 Shell 敲命令	一次性的手工操作；人盯着屏幕
subprocess.run()	脚本里自动执行命令、拿输出、做判断——巡检、批处理的骨架

11.3 脚本的“正式门面”：if __name__ == "__main__"（认识即可）

```
if __name__ == "__main__":  
    main()
```

两行认识（进阶册场景 1、4 的脚本用了它）：① __name__ 是 Python 给每个文件的“名字标签”——文件被直接运行时它是 "__main__"，被别的文件 import 时它是文件名。所以这句话的意思 = **“只有直接跑我这个文件时，才执行下面的 main()”**（被导入时不自动跑，避免误触发）。② 另一种等效写法 raise SystemExit(main())：跑 main 并把它返回值当“退出码”交给系统（0=成功、非 0=失败），相当于 Shell 里的 exit \$?。现在只需混个眼熟——**进阶册场景 1、4 的脚本用了它**（其他脚本直接调 main() 也能跑）。

11.4 章末作业 🌐

1. 用 Python 调 df -h，打印结果。
2. 用 Python 调 free -h，只打印内存那一行。
3. 用 returncode 判断：ls 一个存在和不存在的目录，分别打印“成功”“失败”。
4. 综合：调 df -h /，把 Use% 那列的数字抓出来（提示：split），超过 80 打印警告（用第 2 章的 if）。

✏ **对应练习册：**基础篇阶段 6（题 6-7、6-8）；进阶场景 3（巡检雏形·王牌项目版本 0）；进阶场景 5（paramiko 远程版·选做）

11.5 这章你会了吗？

- ☐ 会写 run() 并拿到 stdout
- ☐ 知道 capture_output 和 text 各管什么
- ☐ 能说出 returncode 为 0 的含义
- ☐ 能说出 subprocess 在运维里的地位（两条线合流的接口）

📖 相关手册：df/free → 《Linux 入门》第 7 章；split → 《Python 内置函数手册》字符串方法速查（11 条）。学完本书 → 开始《Python 练习册：基础篇》。

附录 A · 章末作业参考答案 (含思考过程)

先自己做，再看答案。每道题答案分两部分："怎么做" (代码) + "为什么这么做" (思考过程)。

第 0 章

作业 (无代码)：参考方向——① 自动化重复操作 (批量改名、巡检)；② 调 API 拿数据 (天气、监控指标)；③ 处理日志与数据 (解析、告警)。写自己的三件事即可，无标准答案。

第 1 章

- 自我介绍卡：**`name = "小明"; age = 19; hobby = "学技术"; print(f"我叫{name}, {age}岁, 喜欢{hobby}")`。**为什么：**变量让数据有了名字，f-string 负责把变量塞进句子。
- 两位数倒序：**`num = 37; ge = num % 10; shi = num // 10; print(ge * 10 + shi)`。**为什么：**% 10 取个位 (37 除 10 余 7)，// 10 取十位 (商 3)，倒过来拼 = $7 \times 10 + 3 = 73$ 。
- `type(3.14) → float; len("linux") → 5`。
- 报错应为 `TypeError: can only concatenate str (not "int") to str`——类型不同不能相加。

第 2 章

- 成绩分段：**`input` 读入 → `int()` 转整数 → `if/elif/else` 分档。**为什么：**`input` 返回字符串，不转整数没法比大小；从上到下判断的顺序很关键 (先 `>= 90` 再 `>= 60`，否则 95 会落进"及格")。
- 乘法表：**外层 `for i in range(1,10)`，内层 `for j in range(1, i+1)`，打印 `f"{j}×{i}={i*j}"`。**为什么：**内层范围到 `i+1` 让每行输出到 `i` 为止 (三角形)。
- 猜数字：**`secret = 随机或固定数; while True: 猜 = int(input(...)); count += 1; 相等 break. while True + break = "一直转, 猜到为止"`。**为什么：**次数未知 → 用 `while` 不用 `for`。
- `for i in range(1, 101): if i % 7 == 0: print(i)`。**为什么：**% 取余为 0 = 整除。

第 3 章

- 通讯录：**`dict` 存名字→电话；查用 `.get`；加 = 赋值；删用 `del` 通讯录["名字"]。**为什么：**按名找人用 `dict` 天然合适；`get` 防 `KeyError`。
- set 去重：**`print(len(名单), len(set(名单)))`。**为什么：**`len` 前后对比直接显示重复人数。
- `lst[2:5] → [30, 40, 50]`。**为什么：**索引从 0 数，30 在位置 2；左闭右开所以结尾写 5。
- `for k, v in ports.items(): print(f"{k} 监听 {v} 端口")`。**为什么：**`items()` 一次拿出键和值，两个变量接。

第 4 章

- 三个小函数：**`def double(x): return x * 2; def greet(name): return "你好, " + name; def is_even(n): return n % 2 == 0`。**为什么：**三个函数覆盖三种返回类型——数字、字符串、布尔值；共同的骨架都是"收参数 → 加工 → return 结果"。忘写 `return` 的话调用处会拿到 `None`。
- 计算器：**`def div(a, b): if b == 0: return "除数不能为 0"; return a / b`。**为什么：**用 `return` 把结果交出去；除零要提前挡。
- check_usage：**`def check_usage(p): if p >= 90: return "红色告警" ...`**为什么：**返回字符串让调用方能继续处理 (打印或写日志)。
- 有 `return` 的函数赋值后 `type` 是计算结果类型；只有 `print` 的赋值后是 `NoneType`。**为什么：**`print` 只展示不交出。

第 5 章

1. **工作目录报告**: `for f in os.listdir("."): print(f, os.path.getsize (见手册第 6 组) (f))`。**为什么**: `listdir` 给名字列表, `getsize` 拿大小, 两个 `os` 工具一拼。
2. `t = os.path.getmtime(f); print(f, time.strftime("%Y-%m-%d %H:%M", time.localtime(t)))`。**为什么**: `getmtime` 给时间戳 (机器读), `strftime` 转成人读格式。
3. `for i in [3,2,1]: print(i); time.sleep(1); print("出发")`。**为什么**: `sleep` 制造节奏。
4. `argv[0] = 脚本名, argv[1:] = a b c`。**为什么**: `argv` 是"命令行切开后的完整列表"。

第 6 章

1. `json.dump(通讯录, f, ensure_ascii=False)` 写; `json.load(f)` 读。**为什么**: 不带 `s` 管文件; `ensure_ascii=False` 防中文变 `\uXXXX`。
2. `lines = 0; chars = 0; for line in f: lines += 1; chars += len(line.strip())`。**为什么**: 逐行累计, `strip` 去掉换行符别算进去。
3. `"w"` 打开已存在的文件内容变空; `"a"` 保留原内容。**为什么**: `w` = 覆盖 (Linux 的 `>`) , `a` = 追加 (`>>`) 。
4. 读原文件内容 → 用 `os.path.join` 拼"日期_原名" → `"w"` 写新文件。**为什么**: `join` 拼路径最稳, 手拼斜杠会出错。

第 7 章

1. `try: 读 JSON except FileNotFoundError: print(友好提示); data = {}`。**为什么**: 指定异常捕获, 其他错误照常报。
2. `def safe_div(a, b): try: return a / b except ZeroDivisionError: return "除数不能为 0"`。**为什么**: 把错误变成返回值, 调用方不用 `try`。
3. 对照 7.3 表: `FileNotFoundError` / `KeyError` / `ValueError` 三种最后一行各不相同。**为什么**: 先看最后一行 = 最快定位。
4. 无文件 → `except` 分支; 有文件 → `try` 分支。两条路都走通 = 异常处理写对了。

第 8 章

1. `data = requests.get(url, timeout=10).json(); print(data["current_weather"]["temperature"])`。**为什么**: `get` → `json()` → 层层取值。
2. 经纬度变量化 + `f-string` 打印。**为什么**: 变量化后换城市只改两个数。
3. 参数传 `abc` 会得到 400 或 JSON 解析错误。**为什么**: 状态码/报错告诉我们"请求本身不合法"。
4. `except requests.exceptions.RequestException: print(友好提示)`。**为什么**: 网络错误种类多, 总称类一把接住。

第 9 章

1. `add_argument("a")`、`add_argument("op")`、`add_argument("b")` 三个位置参数, `if` 判断 `op` 再运算。**为什么**: 位置参数按敲的顺序进 `args`。
2. `add_argument("cmd"); if args.cmd == "add": ... elif "find": ...` **为什么**: 一个位置参数当命令名, `if` 分支分发——这就是"子命令"的雏形。
3. `add_argument("--threshold", type=int, default=80)`。**为什么**: 可选参数配默认值 = 不传也能跑。
4. `--help` 自动列出 `usage`、参数表。`argparse` 与 Linux `--help` 同源。

第 10 章

1. `logging.basicConfig(level=logging.INFO, format="%asctime)s %(levelname)s %(message)s")`; 检查完 `logging.info(f"使用率 {usage}%")`。**为什么**: `basicConfig` 全局配置一次, 之后随意记。
2. `basicConfig` 加 `filename="app.log"`。**为什么**: 日志落文件 = 可回查。
3. `level=DEBUG` 会多出 `debug` 行; `level=ERROR` 只显示 `error`。**为什么**: 级别是过滤网, 拦掉更低级。
4. `except Exception as e: logging.error(f"失败: {e}")`。**为什么**: 错误信息进日志, 以后排查有据。

第 11 章

1. `r = subprocess.run(["df", "-h"], capture_output=True, text=True); print(r.stdout)`。**为什么**: `capture_output` 抓输出, `text` 转字符串。
2. `lines = r.stdout.split("\n"); print(lines[1])`。**为什么**: 第 1 行是表头, 数据从第 2 行起。
3. `returncode == 0` → "成功", 否则 "失败"。**为什么**: 0 成功非 0 失败, 比读输出文字可靠。
4. 取 `lines[1].split()[-2]` (倒数第二列是 `Use%` 值, 注意带 `%` 号要先去掉再比较)。**为什么**: `split` (第 1 章学过) 后按位置取列; 带 `%` 号不能直接比大小, 参考 `tr -d` 的思路处理。

附录 B · 术语表 (中英对照)

术语	含义
交互式模式	敲一行出一行结果的模式 (提示符 >>>)
脚本 (.py 文件)	存成文件整体运行的代码, 可保存可复用
变量 variable	贴标签的盒子, 装数据
类型 type	盒子装的什么货 (int/float/str 等)
字符串 string	文字, 引号包着的一切
f-string	带 f 的字符串, {变量} 塞值
整除 // 与取余 %	商去小数 / 余数
布尔值 boolean	True / False, 条件的答案
缩进 indent	行首空格, Python 的"括号"
循环 loop	for (知道次数) / while (不知道次数)
break / continue	跳出循环 / 跳过本轮
列表 list	一排盒子, 索引从 0 数, 可增删改
切片 slice	[a:b] 取一段, 左闭右开
字典 dict	键值对, 按名找值
元组 tuple	不可修改的列表
集合 set	去重机, 不保顺序
函数 function	打包好的可复用代码块 (def)
参数 argument	函数/命令的输入
return 返回	把结果交出去 (区别于 print 展示)
作用域 scope	变量有效的区域
模块 module / 导入 import	工具箱 / 搬工具箱
标准库	Python 自带的工具箱集合 (os、time、json...)
异常 exception	程序运行时的错误
Traceback	报错时打印的调用链, 从下往上读
try / except	尝试 / 接住错误
finally	无论成败都执行的收尾块
HTTP 请求/响应	向网址要数据 / 对方返回数据
状态码 status code	200 成功、404 不存在、500 服务器错
超时 timeout	等多久无响应算失败
命令行参数	敲命令时传入的参数 (argparse 管理)
日志级别	DEBUG < INFO < WARNING < ERROR
返回码 returncode	命令成绩单: 0 成功, 非 0 失败
子进程 subprocess	Python 调系统命令的接口
stdout	命令的标准输出 (抓回来当数据)
API	Application Programming Interface, 应用程序接口: 程序之间"说话的接口", 你按约定发请求、对方按约定回数据 (requests 调的就是 API)

附录 C · 学习路线图 (第 0~11 章共 12 章顺序 + 每章耗时)

章节	耗时	学完的产出 / 里程碑
第 0 章 Python 与 AI 运维的关系	1h	第一行代码
第 1 章 print、变量、类型	1.5h	自我介绍卡、两位数倒序
第 2 章 if、for、while	2h	成绩分段、猜数字
第 3 章 list、dict、tuple、set	2.5h	通讯录小程序
第 4 章 def、参数、返回值	2h	三个小函数、计算器
第 5 章 import、os、time	1.5h	工作目录报告
★ 检查点①	—	学完第 5 章后，能独立完成第 5 章全部章末作业和自测清单（逐项运行、逐项打勾）；这就是本书的第一个基础检查点。
第 6 章 open/with、json	2h	通讯录存 JSON
第 7 章 try/except/finally	1.5h	★ 检查点②：带 try/except 的脚本
第 8 章 requests	2h	天气查询器雏形（脚本 2 核心）
第 9 章 argparse	1.5h	python3 weather.py 杭州
第 10 章 logging	1h	带日志的脚本
第 11 章 subprocess	1.5h	★ 通往 server-health-check 的接口
合计	20h	学完接《Python 练习册：基础篇》

学习顺序：本书在 Linux 三连之后开始。两件成品脚本：① 批量文件重命名（第 5、6 章知识）② 天气查询器（第 8、9 章知识）——练习册进阶篇会把它们作为收尾项目。

附录 D • 函数与模块速查

类别	常用内容	用途
输入输出	print / input	显示信息、读取输入
类型与序列	type / int / str / len / range / enumerate	检查、转换、计数、范围和编号遍历
数据结构	list / dict / tuple / set	保存和组织数据
文件与 JSON	open / with / json.load / json.dump	读写文本和结构化数据
自动化模块	os / time / requests / argparse / logging / subprocess	路径、时间、接口、参数、日志和系统命令

需要按函数查用法时，查《Python 内置函数手册》对应小节。

附录 E • venv 与 pip：给项目一个独立工具箱

为什么需要它：系统自带的 Python 是“公家的”——用 apt 装的包全机器共用，装坏了会影响系统里所有程序。而你的项目需要“自己的”工具箱：装上什么、什么版本，都由这个项目说了算，换台机器还能照单重建。

venv（虚拟环境）= 项目专属工具箱；pip = 往箱子里装工具的手。两条一起用，是 Python 世界最通用的做法。

E.1 三步上手（先记住这三条）

```
$ sudo apt install -y python3-venv python3-pip # ① 一次性准备
$ cd ~/lab/myproject
$ python3 -m venv .venv # ② 在项目目录里建工具箱（.venv 是惯例名）
$ source .venv/bin/activate # ③ 激活（提示符前出现 (.venv) 就对了）
(.venv) $ pip install requests # 之后装的包只进这个箱子
(.venv) $ python -c "import requests; print(requests.__version__)"
```

⚠ **实测提醒：**跳过第 ① 步直接建环境，目录虽然能建出来，但里面的 pip 是缺的（Ubuntu 把引导组件拆成了独立包）——进箱后敲 pip 会报 **No module named pip**。先 apt 装 python3-venv python3-pip，再建环境。

E.2 日常四条（够用）

命令	作用	什么时候用
deactivate	退出工具箱，回到系统环境	这个项目干完了
source .venv/bin/activate	重新进入	下次开终端接着做这个项目时（ 每次都要先激活 ）
pip list	看箱子里装了什么	想确认装没装、装的哪版
pip uninstall 包名	卸掉某个包	装错了、装重了

判断小技巧：敲 which python——显示 .venv/bin/python 说明在箱子里；显示 /usr/bin/python3 说明还在系统里。

E.3 交付给别人：requirements.txt（两行）

```
(.venv) $ pip freeze > requirements.txt # 导出清单：这个项目需要哪些包、什么版本
$ pip install -r requirements.txt # 在别人的机器上照单重建，版本一致
```

把 requirements.txt 交给同事/提交到仓库，对方一条命令就能还原出和你一样的环境——这就是“环境可复制”。

E.4 什么时候用 apt，什么时候用 pip

要装的东西	用什么	例子
系统工具 / 服务	<code>sudo apt install</code>	htop、nginx、curl、tree
Python 第三方库	venv + pip	requests、paramiko、pandas
系统自带的 Python 库（老办法）	<code>sudo apt install python3-xxx</code>	python3-requests（能用，但不隔离、版本老）

记忆点：给系统装东西用 apt，给项目装东西用 pip（且先激活 venv）。

E.5 两个常见报错（人话版）

报错	什么意思	怎么解决
error: externally-managed-environment	系统拦住了你：不允许直接往系统 Python 里装包（Ubuntu 24.04 起默认如此，26.04 同样）	先建并激活 venv，再 <code>pip install</code> ；确实要给系统装就用 <code>apt install python3-xxx</code>
bash: pip: command not found	两个可能：箱子没激活，或没装 python3-pip	先 <code>source .venv/bin/activate</code> ；仍然没有就 <code>sudo apt install python3-pip python3-venv</code>

E.6 小练习（做完你就掌握了）

① 在 `~/lab` 下建一个项目目录，创建并激活虚拟环境；② 装上 `requests`，写三行代码请求一次 `https://example.com` 并打印状态码；③ 导出 `requirements.txt`；④ 把 `.venv` 整个删掉，再用 `requirements.txt` 重建一次并跑通同一段代码。

📌 一条纪律：`.venv` 不要提交到 Git（体积大、且随时能重建）——提交 `requirements.txt` 就够了。这条在你学 Git 时会用到。